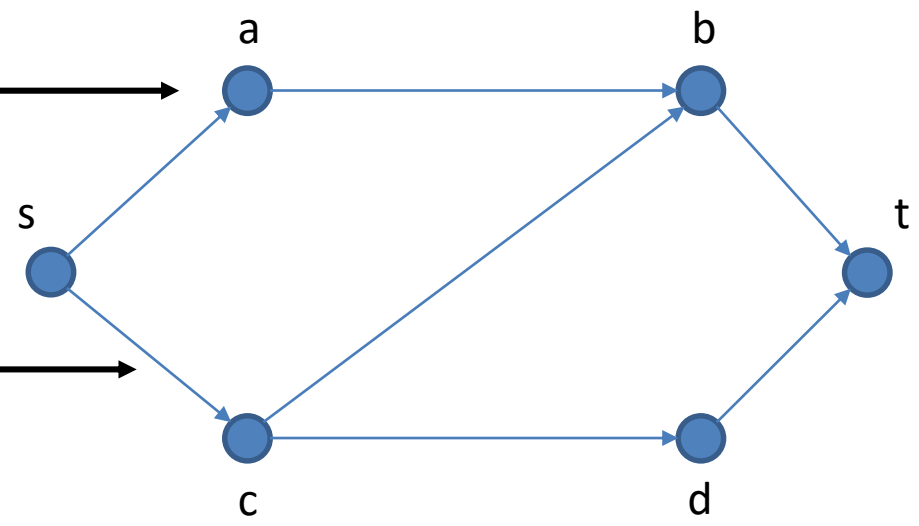
Unidade IV
Fluxo em grafos

O que são redes?

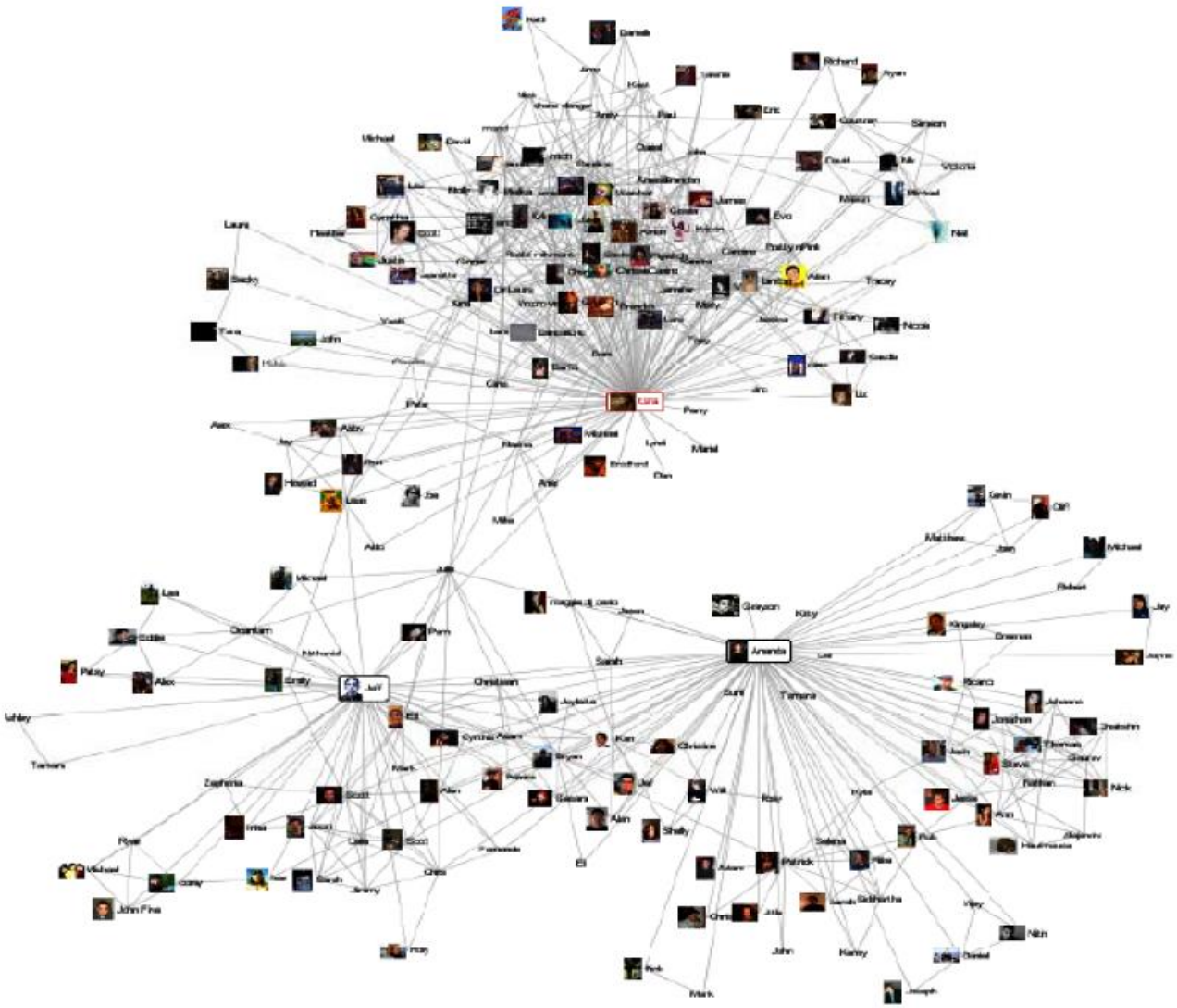
Nó ou Vértices

Linhas ou Arestas



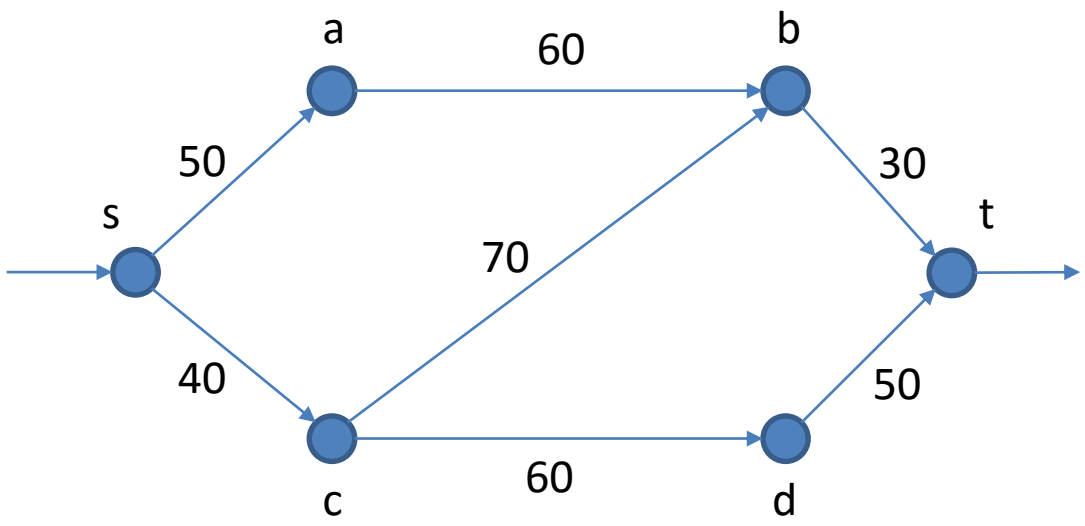
Rede = grafo

Rede Social



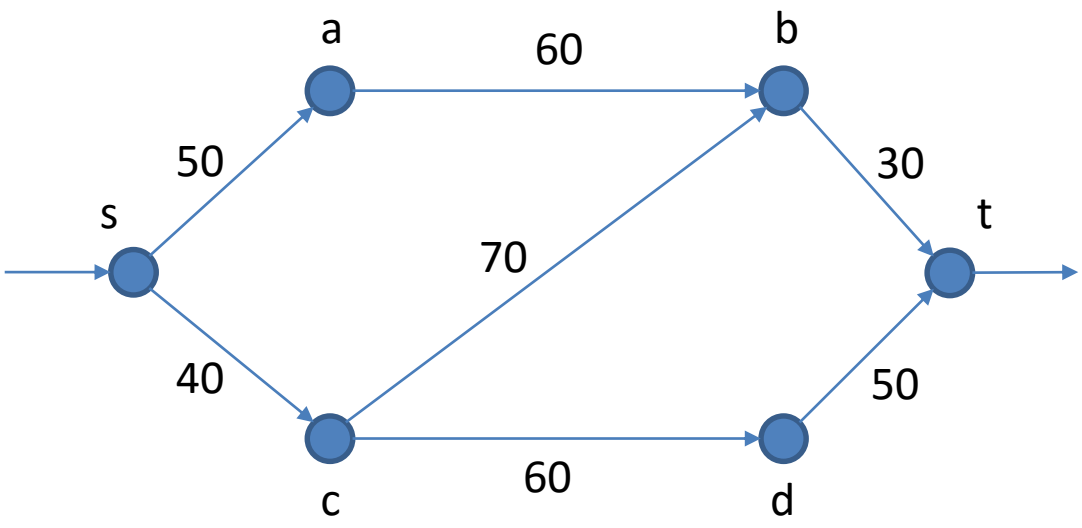
Fonte: **"Vizster: Visualizing Online Social Networks."** Jeffrey Heer and danah boyd. *IEEE Symposium on Information Visualization (InfoViz 2005)*.

Fluxo em grafos



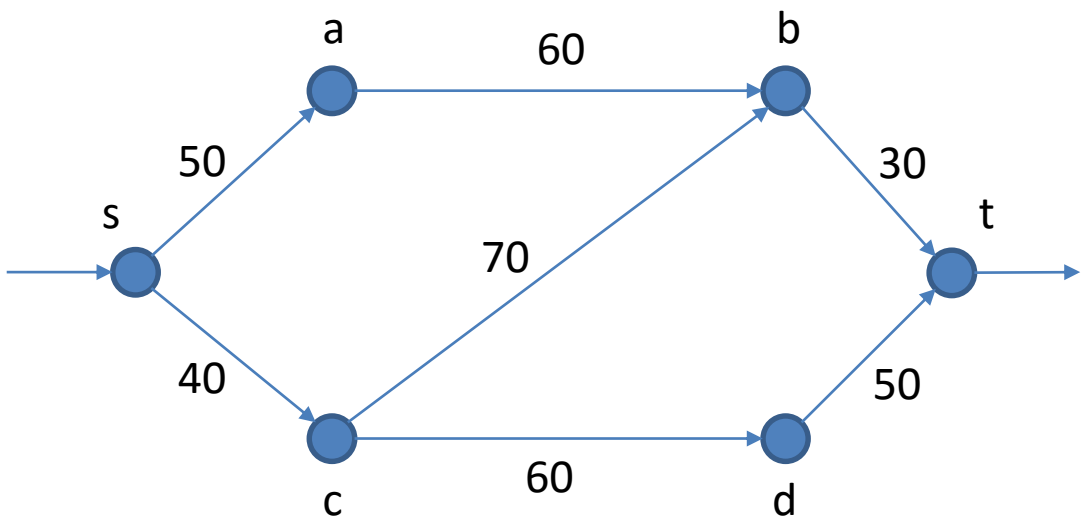
- Exemplo de uma malha rodoviária.
- Os ‘pesos’ das arestas mostram o número máximo de veículos que podem passar pelos arcos numa determinada unidade de tempo.
- Estes números são chamados de **capacidades**.

Fluxo em grafos



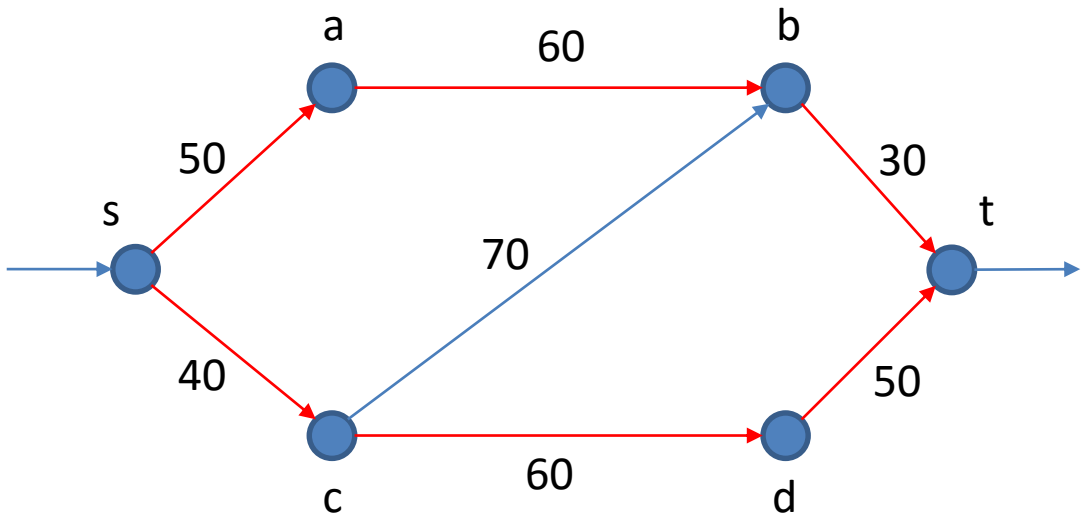
- s: fonte emissor de fluxo.
- t: terminal, quem consome o fluxo

Fluxo em grafos



- Qual a capacidade máxima de veículos nesta rede? Ou seja, quanto veículos conseguem sair de s e chegam a t?
- 90 ou 80?
- Resposta:

Fluxo em grafos



- Qual a capacidade máxima de veículos nesta rede? Ou seja, quanto veículos conseguem sair de s e chegam a t?
- 90 ou 80?
- Resposta: 70

Fluxo em grafos

- Exemplos:
  - Valores (pesos de arestas) estáticos:
    - A distância da rodovia que liga uma cidade a outra.
  - Valares dinâmicos
    - Na rodovia passam 10 carros por minutos.

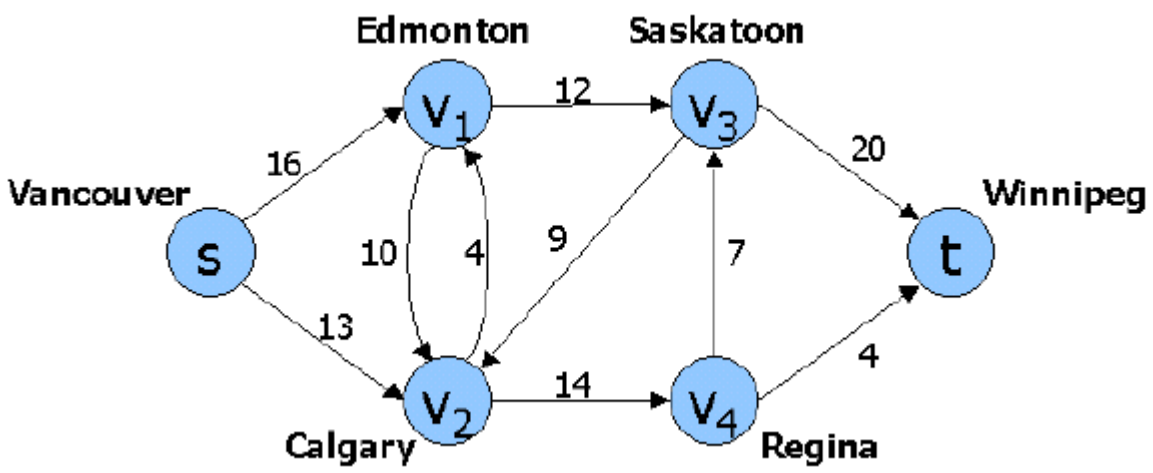
- Fluxo em grafos
  - Quantos veículos passam por minuto em uma rodovia?

Fluxo em grafos

- Recurso que move por meio do arco de um grafo.
- Exemplos:
  - Líquidos em canos;
  - Trafego em rede de computadores
  - Veículos e rodovias
  - Etc...

Fluxo em grafos

- Considere um material percorrendo um sistema deste a origem, onde é produzido, até o destino, onde é consumido
- Exemplos de fluxos: líquidos em tubulação, peças em linhas de montagem, corrente em redes elétricas, informações em redes
- Exemplo: Fábrica em Vancouver (s) e depósito em Winnipeg (t)



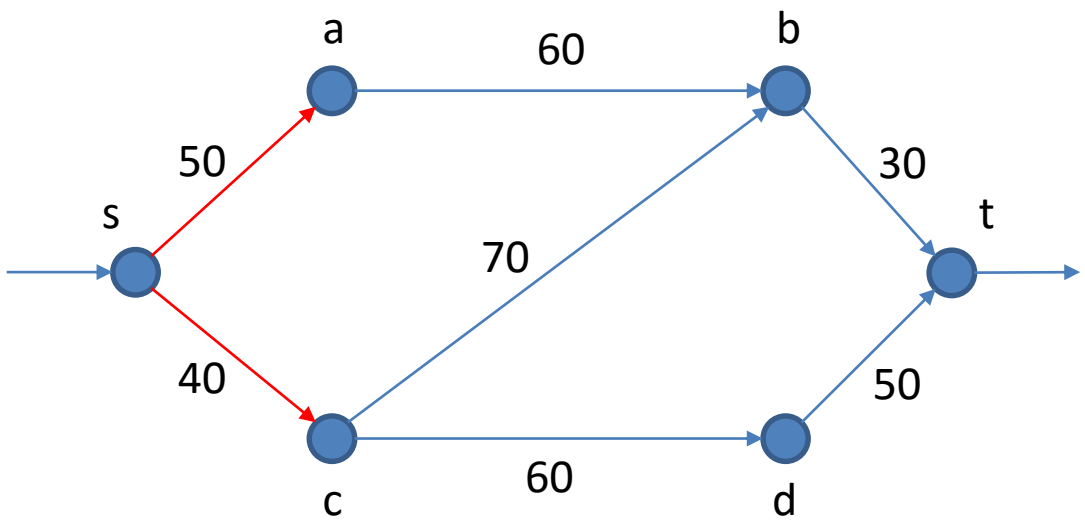
Fluxo em grafos

- Um Fluxo em Rede G = (V, E) é um grafo dirigido em que cada aresta $(u, v) \in E$ possui uma capacidade associada $c(u, v) \geq 0$.
- Se $(u, v) \notin$ G, consideramos que $c(u, v) = 0$
- Destacam-se os vértices: s, a origem, e t, o destino
- Consideramos que todos os outros vértices estejam em algum caminho de s para t
- Um Fluxo em G é uma função de fluxo, tal que:
  - Restrição de capacidade: $\forall\, u, v : f(u, v) \leq c(u, v)$
  - Conservação de fluxo: todo fluxo que chega a um vértice do grafo sai dele. Ou seja, todo $u \in V - \{s, t\}$.

O problema do fluxo máximo

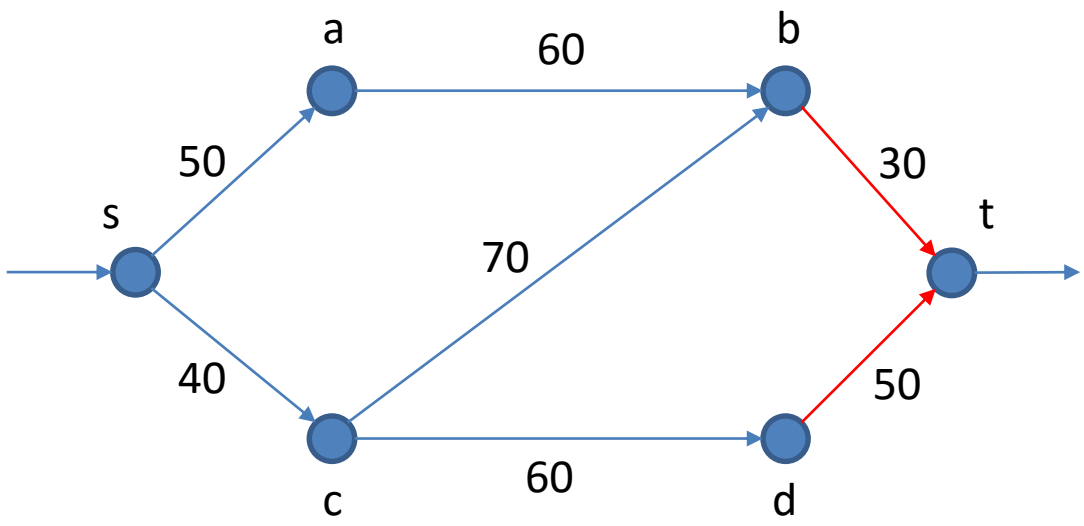
- **Corresponde a achar a maior quantidade de fluxo que pode passar por um dado grafo**.
- Para a formulação do problema é importante conhecer a noção de corte:
  - Seja G = (V, E) um grafo com fluxo e seja $X \subset V$ tal que $s \in X$ e $t \notin X$. Então um corte $K = (X, V - X)$ em G é um conjunto de arcos de G com extremidade inicial em X e extremidade final em V – X (complemento de X em relação a V).

O problema do fluxo máximo



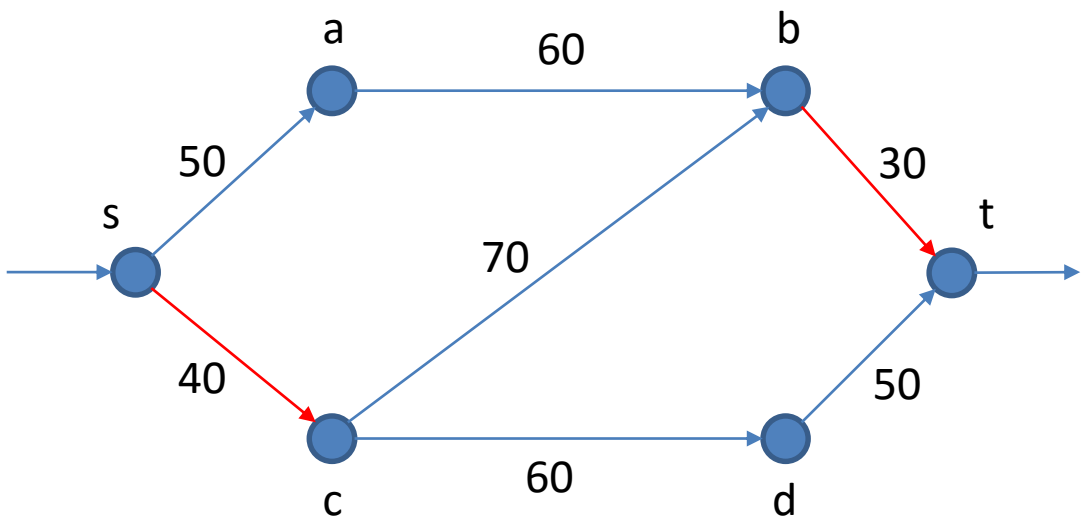
- O conjunto {sa, sc} é um corte com capacidade 90.
- X = {s} e V – X = {a, b, c, d, t}

O problema do fluxo máximo



- O conjunto {bt, dt} é um corte com capacidade 80.
- X = {s, a, b, c, d} e V – X = {t}

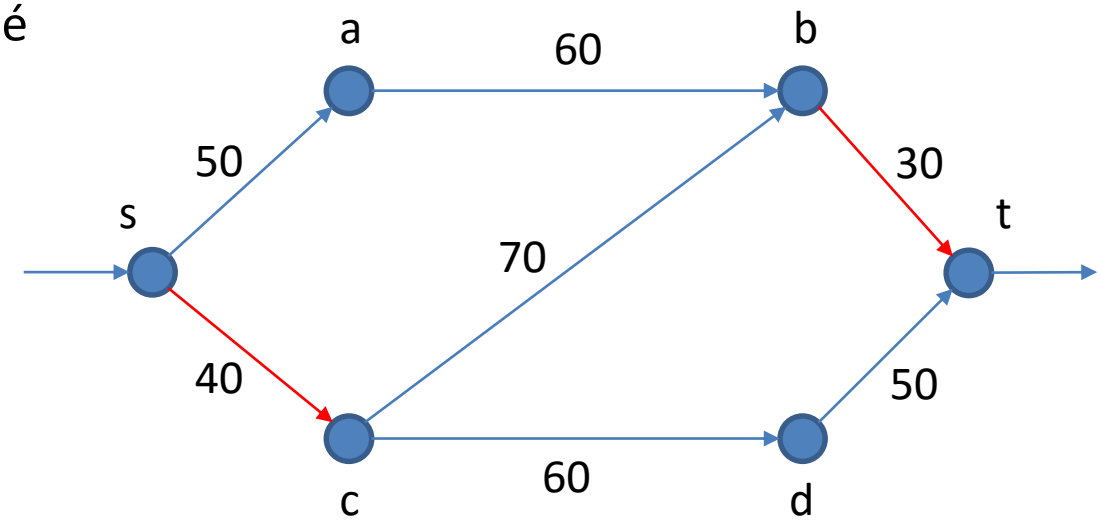
O problema do fluxo máximo



- O conjunto {sc, bt} é um corte com capacidade 70.
- X = {s, a, b} e V – X = {c, d, t}

O problema do fluxo máximo

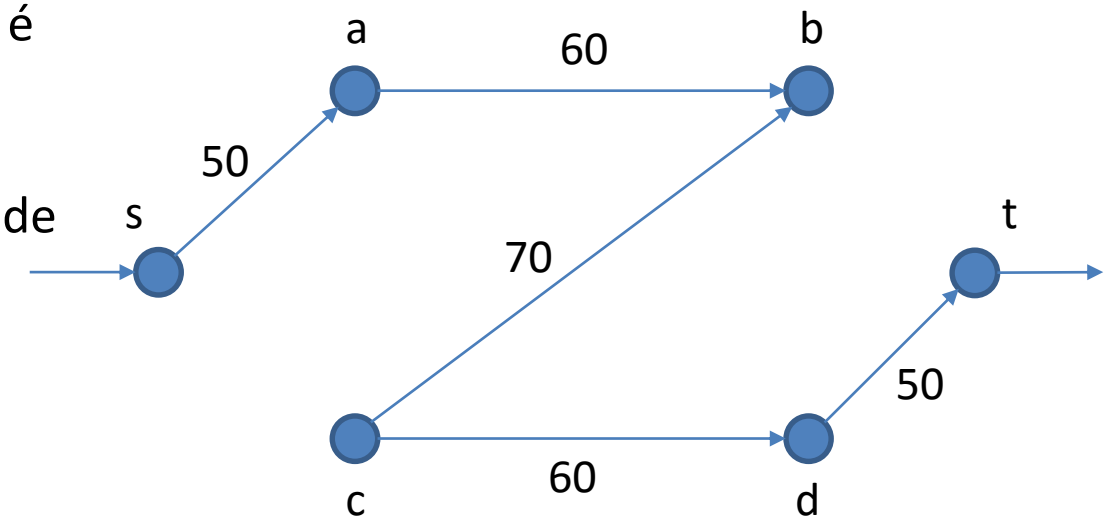
Por quê {sc, bt} é um corte?



- O conjunto {sc, bt} é um corte com capacidade 70.
- X = {s, a, b} e V – X = {c, d, t}

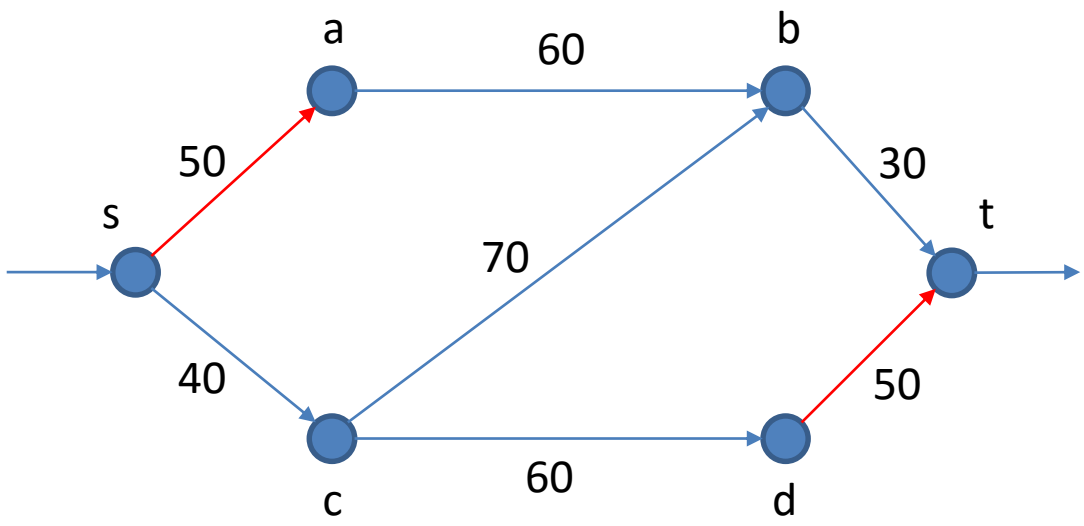
O problema do fluxo máximo

Por quê {sc, bt} é um corte? Sua retirada impede o fluxo de ir de s para t



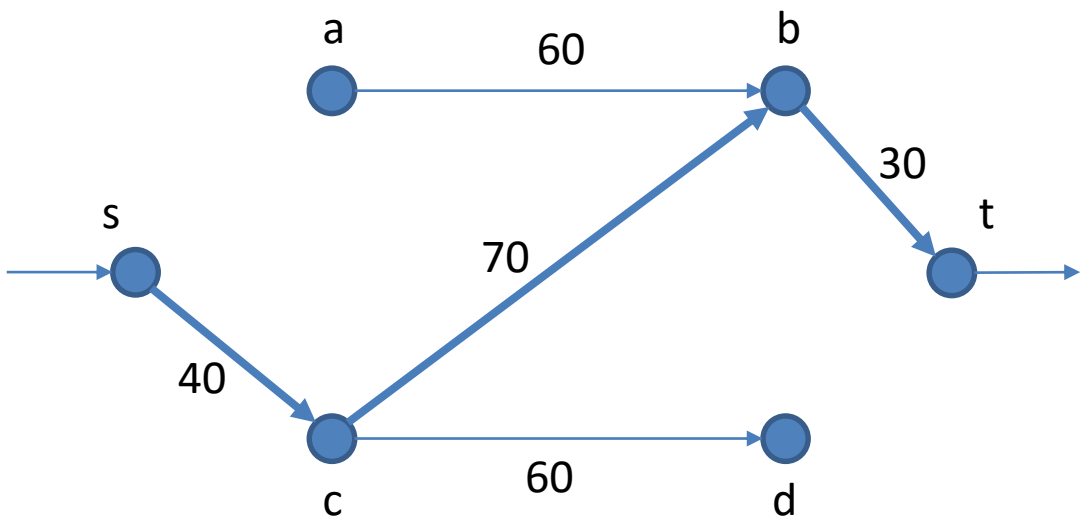
- O conjunto {sc, bt} é um corte com capacidade 70.
- X = {s, a, b} e V – X = {c, d, t}

O problema do fluxo máximo



- O conjunto {sa, dt} **não** é um corte, pois sua retirada não impede o fluxo de ir de s para t por meio do caminho scbt.

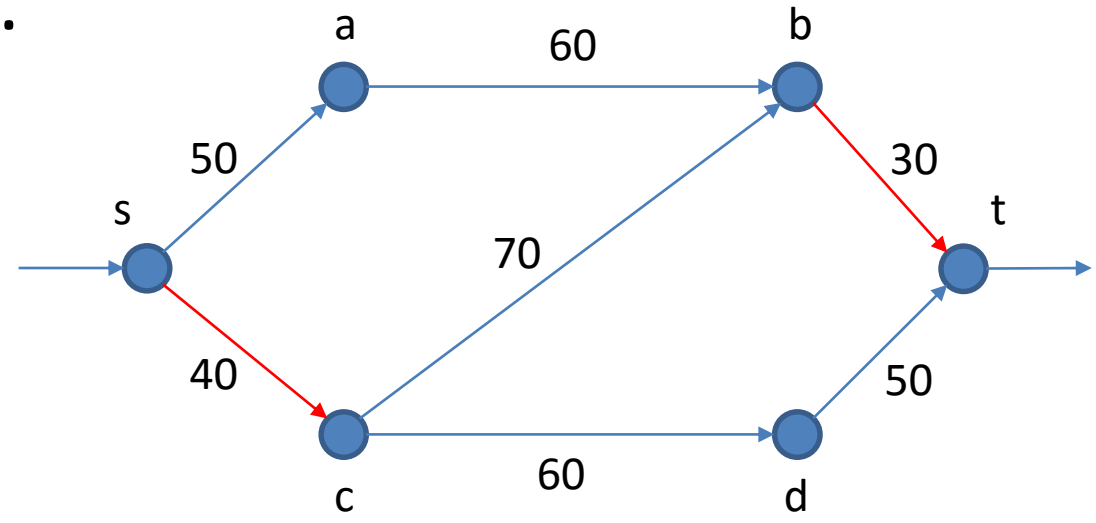
O problema do fluxo máximo



- O conjunto {sa, dt} **não** é um corte, pois sua retirada não impede o fluxo de ir de s para t por meio do caminho scbt.

O problema do fluxo máximo

- O menor corte é o conjunto {sc, bt}, que é um corte com capacidade 70.
- Veja que este também é o fluxo máximo do grafo.



O problema do fluxo máximo

- O menor corte é o conjunto {sc, bt}, que é um corte com capacidade 70.
- Veja que este também é o fluxo máximo do grafo.
- Coincidência? Claro que não.
- Teorema de Ford e Fulkerson

Teorema de Ford e Fulkerson

- Se partimos de um fluxo total nulo teremos, logo de início, que este fluxo terá sempre valor menor do que a capacidade de qualquer corte.
- Podemos aumentar este fluxo aos poucos, aumentando e testando para comparar o seu valor comas as capacidades dos cortes.
- Em um dado momento, ele se tornará igual a uma das capacidades, e é claro que isso acontecerá com a menor de todas capacidades.

Teorema de Ford e Fulkerson

- Portanto, um corte cuja capacidade possa se tornar igual ao valor de um fluxo é um corte de **mínima capacidade**.
- Veja que **o fluxo não pode mais aumentar**, pois ele passa por todo o corte, e este já estará saturado.
- Portanto, este fluxo será **máximo**.

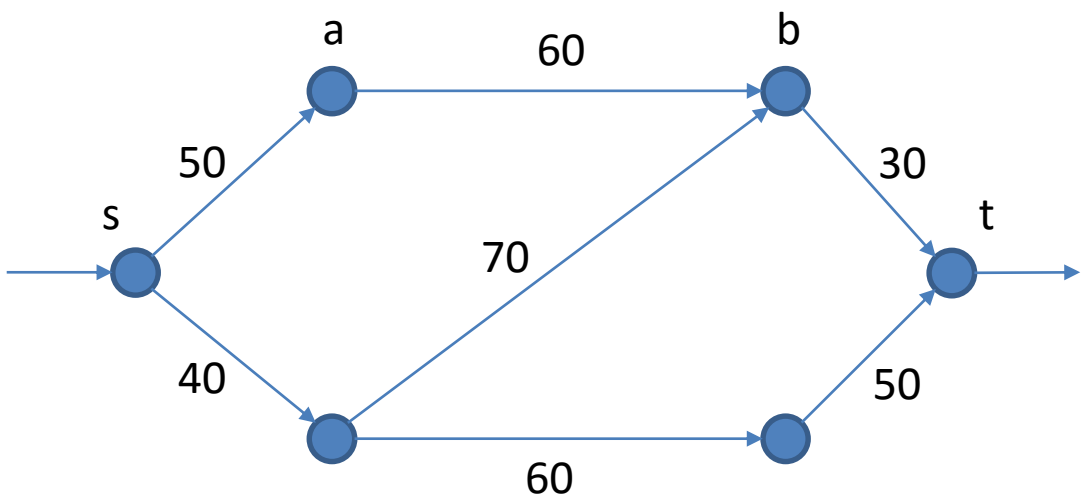
Teorema de Ford e Fulkerson

- Chegamos então ao teorema de Ford e Fulkerson conhecido como ***max flow-min cut***:

O valor do fluxo máximo em um grafo é igual à capacidade do corte de capacidade mínima.

O problema do fluxo máximo

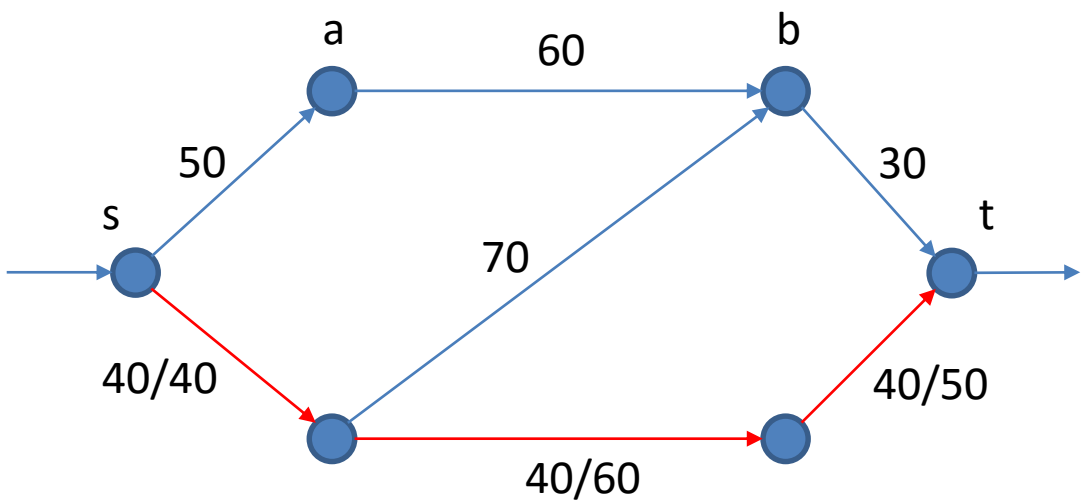
- Intensidade do fluxo:



- O fluxo neste grafo tem intensidade 70.

O problema do fluxo máximo

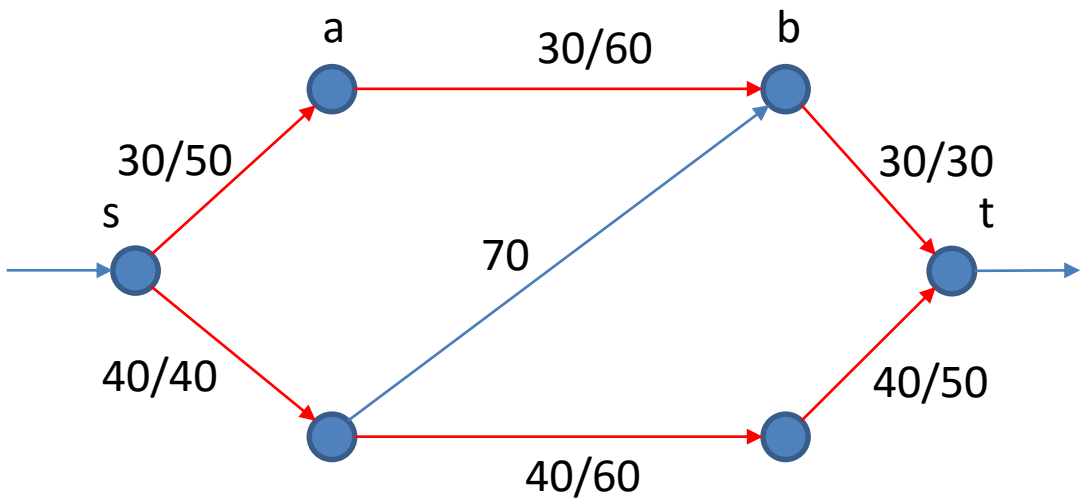
- Intensidade do fluxo:



- O fluxo neste grafo tem intensidade 70.

O problema do fluxo máximo

- Intensidade do fluxo:



- O fluxo neste grafo tem intensidade 70.

Algoritmo de Ford e Fulkerson

- Dado um dígrafo capacitado e um fluxo que respeita a capacidade dos arcos, dizemos que um aro u,v está cheio se o fluxo no arco é igual a sua capacidade.
- Assim, em um caminho de s para t no grafo, se nenhum arco do caminho está cheio, então podemos chamá-lo de **Caminho de Aumento**

Algoritmo de Ford e Fulkerson

- Redes Residuais:
  - Dado um dígrafo capacitado com vértice inicial e final, e um fluxo f que respeita sua capacidade c.
  - capacidade residual de (u, v) é a quantidade de fluxo adicional que podemos enviar de u para v sem ultrapassar a sua capacidade. Ou seja:

$$c(a) - f(a)$$

  - Uma rede residual consiste em arestas que podem admitir mais fluxo.
  - A capacidade residual do caminho de aumento é a menor capacidade residual do seus arcos

Algoritmo de Ford e Fulkerson

- Caminho de Aumento:
  - Seja G = (V, E) um grafo e f um fluxo em G.
  - Um caminho de aumento p é um caminho simples de s para t em G
  - A capacidade residual de p é a capacidade máxima pela qual podemos aumentar o fluxo em cada aresta de um caminho de aumento.

Algoritmo de Ford e Fulkerson

- Cada iteração do algoritmo começa com um fluxo f que respeita a capacidade. Na primeira iteração o fluxo é zero.
- Entrada: Grafo G = (V,E), vértices s e t
- Saída: Função de fluxo f
- Algoritmo:

Na primeira vez irá existir pois o fluxo iniciou em zero, portanto há como aumentar

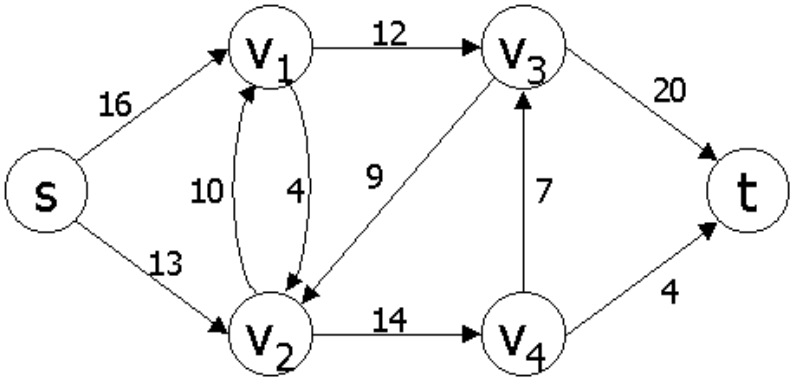
Iniciar fluxo f como 0 (para toda aresta f(u,v) = 0)
while Existir caminho de aumento p de s para t em G **do**
Encontre um caminho de aumento p
Calcule a capacidade residual de p
Ampliar fluxo f ao longo de p
end while
return f

Na primeira vez irá existir

a menor capacidade residual dos arcos de p (na primeira vez será o valor do arco de menor capacidade no caminho)

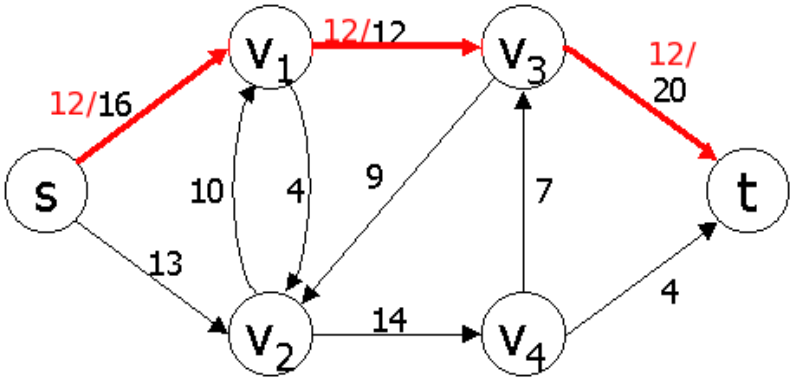
Algoritmo de Ford e Fulkerson

Iniciar fluxo f como 0 (para toda aresta f(u,v) = 0)
while Existir caminho de aumento p de s para t em G **do**
Encontre um caminho de aumento p
Calcule a capacidade residual de p
Ampliar fluxo f ao longo de p
end while
return f



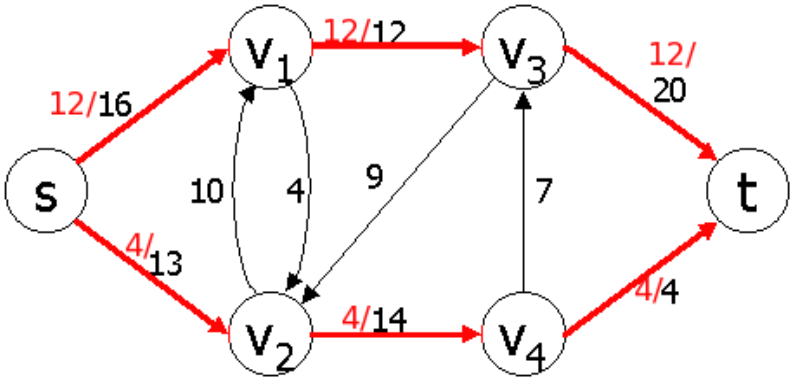
Algoritmo de Ford e Fulkerson

Iniciar fluxo f como 0 (para toda aresta f(u,v) = 0)
while Existir caminho de aumento p de s para t em G **do**
Encontre um caminho de aumento p
Calcule a capacidade residual de p
Ampliar fluxo f ao longo de p
end while
return f



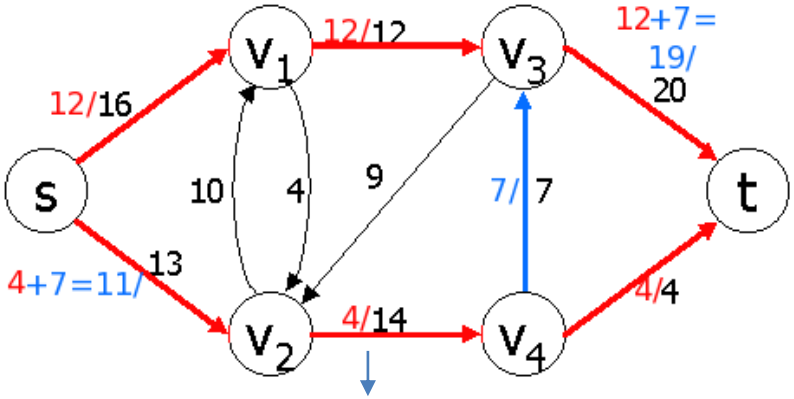
Algoritmo de Ford e Fulkerson

Iniciar fluxo f como 0 (para toda aresta f(u,v) = 0)
while Existir caminho de aumento p de s para t em G **do**
Encontre um caminho de aumento p
Calcule a capacidade residual de p
Ampliar fluxo f ao longo de p
end while
return f



Algoritmo de Ford e Fulkerson

Iniciar fluxo f como 0 (para toda aresta f(u,v) = 0)
while Existir caminho de aumento p de s para t em G **do**
Encontre um caminho de aumento p
Calcule a capacidade residual de p
Ampliar fluxo f ao longo de p
end while
return f



4+7=11/14

Algoritmo Menor Caminho

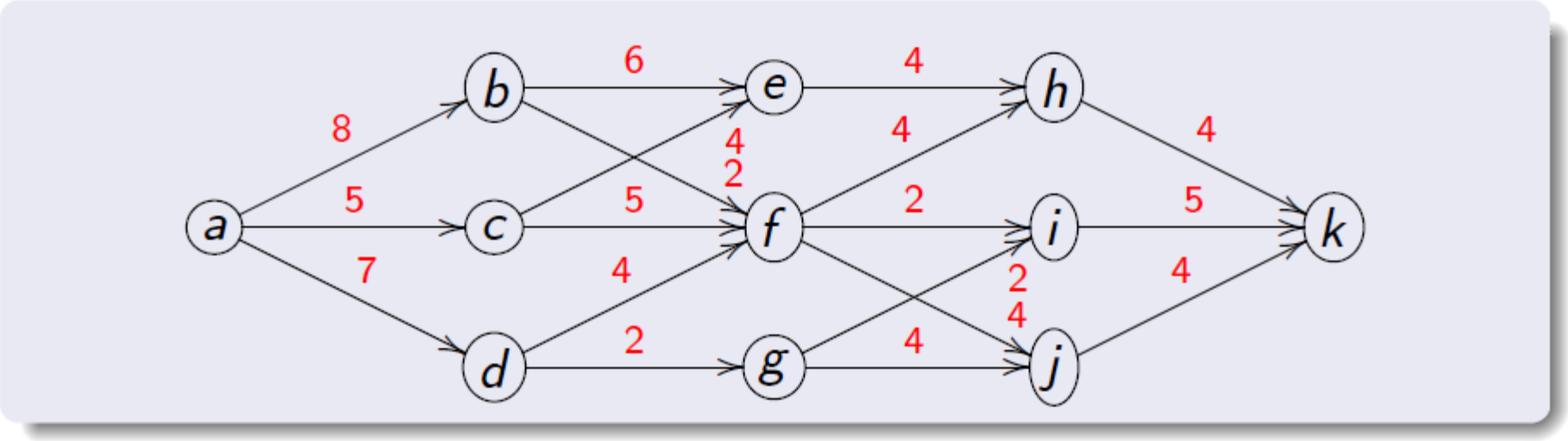
- Problema
  - dado o grafo G=(V, E) orientado e com distâncias $c_{i,j}$ associado ao arco (i, j).
  - Problema: Obter o caminho mais curto entre dois vértices s e t.

Algoritmo Menor Caminho

- Comprimento
  - O comprimento de um caminho é igual à soma dos comprimentos (distâncias) dos arcos que formam o caminho.
  - A distância ou comprimento de um arco pode ter diversas interpretações dependendo da aplicação: custos, distâncias, consumo de combustível, etc.

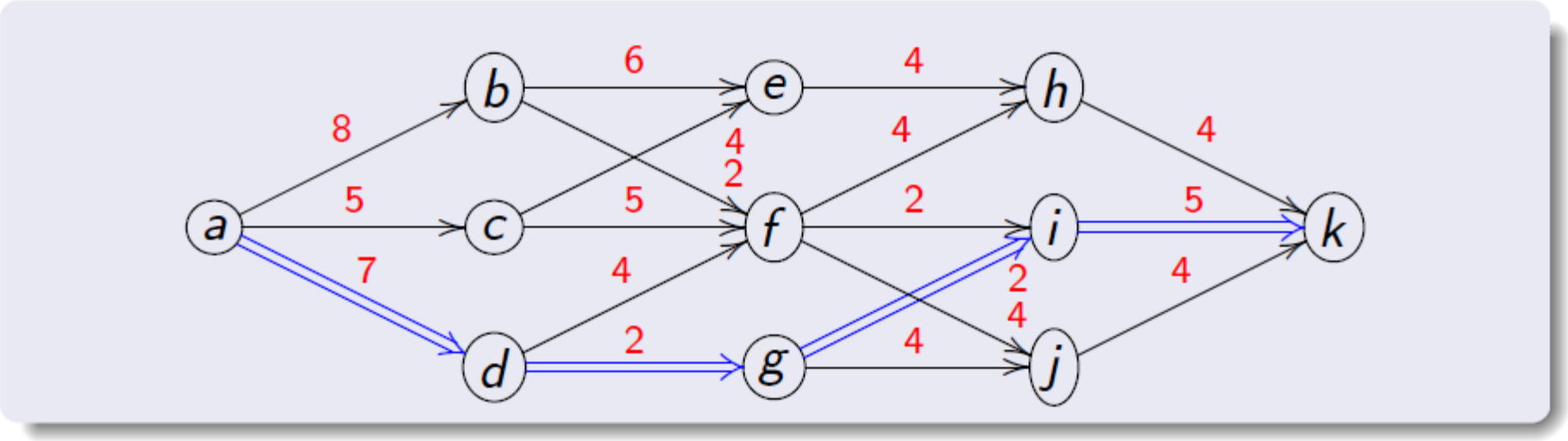
Algoritmo Menor Caminho

- Exemplo
  - Dado um mapa rodoviário, determinar a rota mais curta de uma cidade a outra (rota mais rápida, rota com menor consumo de combustível, rota com menor valor de pedágio)



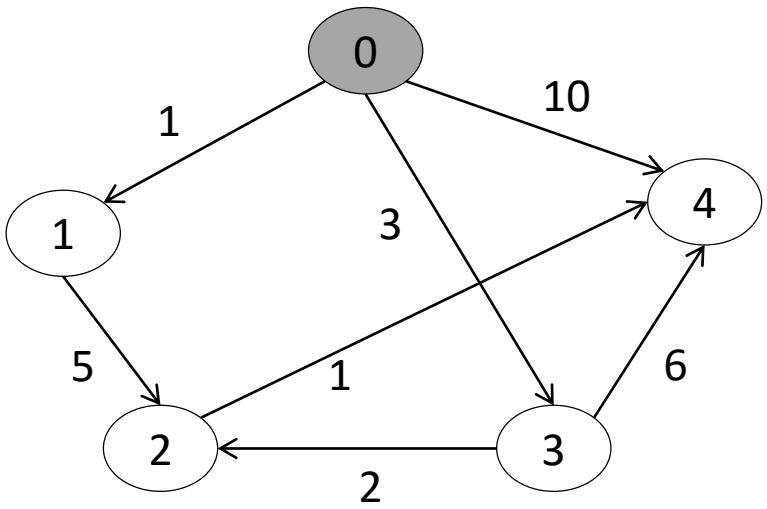
Algoritmo Menor Caminho

- Exemplo
  - Dado um mapa rodoviário, determinar a rota mais curta de uma cidade a outra (rota mais rápida, rota com menor consumo de combustível, rota com menor valor de pedágio)



Dijkstra

a)



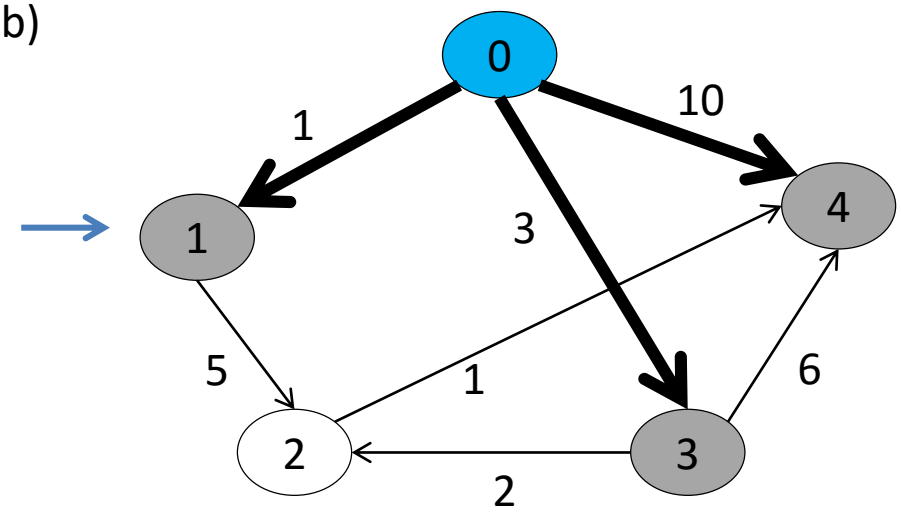
Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(a)	$\emptyset$	∞	∞	∞	∞	∞
(b)	$\{0\}$	0	1	∞	3	10
(c)	$\{0,1\}$	0	1	6	3	10

Dijkstra

Para saber qual o próximo vértice a ser explorado, verifica qual o vértice de menor caminho até o momento (não finalizado).

O algoritmo usa uma estratégia gulosa: sempre escolher o vértice mais leve (ou o mais perto)

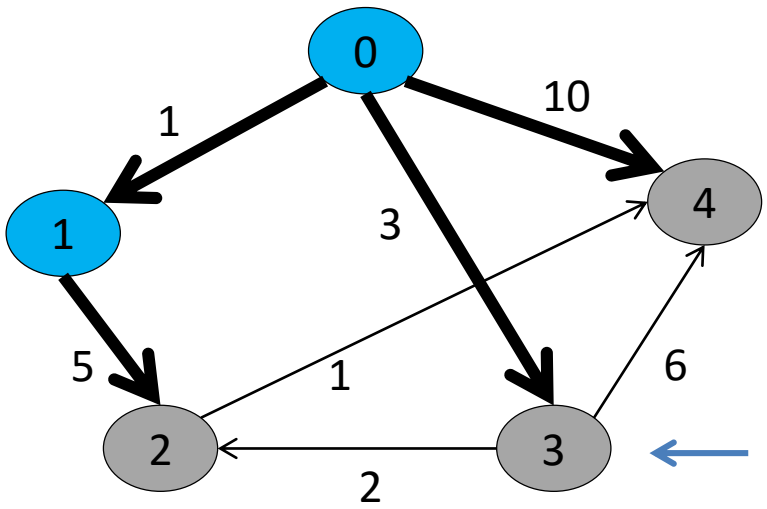
b)



Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(a)	$\emptyset$	∞	∞	∞	∞	∞
(b)	$\{0\}$	0	$\boxed{1}$	∞	3	10
(c)	$\{0, 1\}$	0	1	6	3	10

Dijkstra

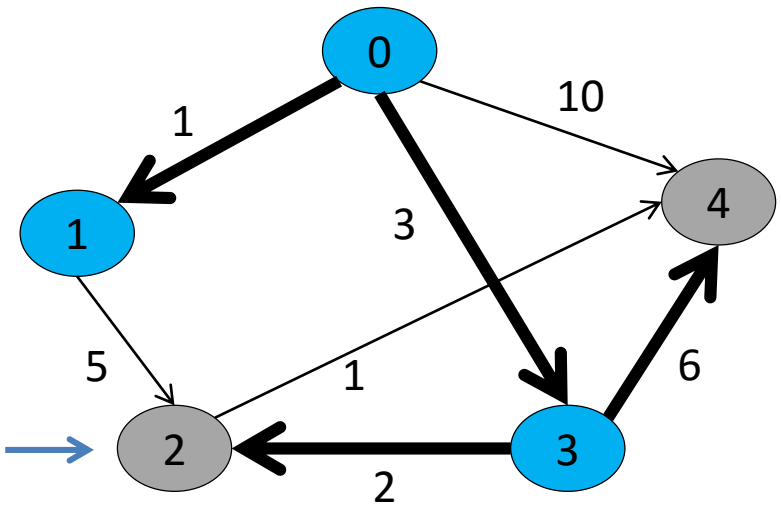
c)



Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(a)	$\emptyset$	∞	∞	∞	∞	∞
(b)	$\{0\}$	0	1	∞	3	10
(c)	$\{0, 1\}$	0	1	6	$\boxed{3}$	10

Dijkstra

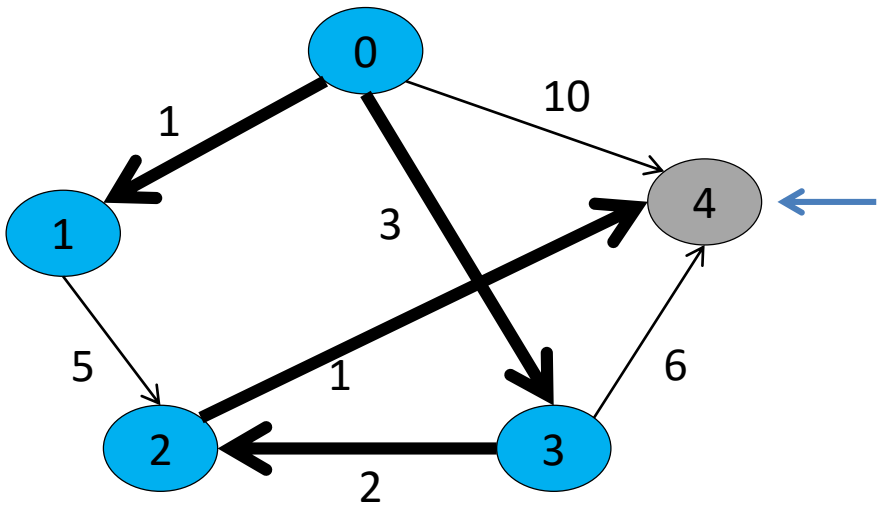
d)



Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(d)	$\{0, 1, 3\}$	0	1	$\boxed{5}$	3	9
(e)	$\{0, 1, 3, 2\}$	0	1	5	3	6
(f)	$\{0, 1, 3, 2, 4\}$	0	1	5	3	6

Dijkstra

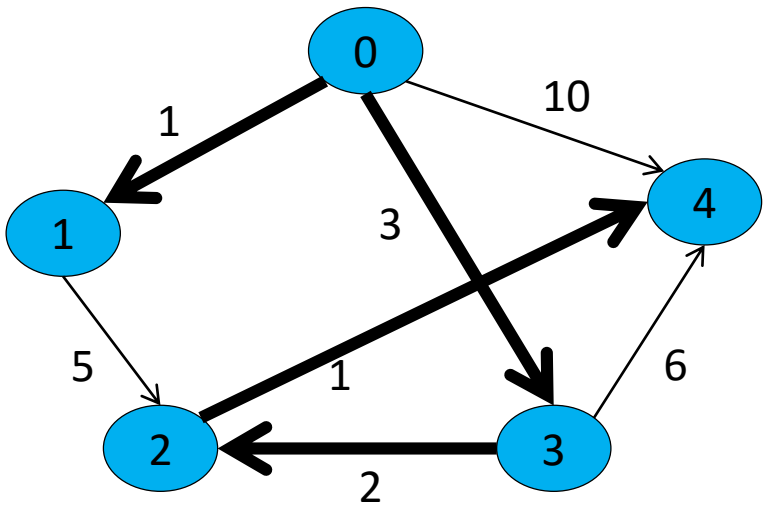
e)



Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(d)	$\{0,1,3\}$	0	1	5	3	9
(e)	$\{0,1,3,2\}$	0	1	5	3	$\boxed{6}$
(f)	$\{0,1,3,2,4\}$	0	1	5	3	6

Dijkstra

f)



Iteração	S	d[0]	d[1]	d[2]	d[3]	d[4]
(d)	$\{0,1,3\}$	0	1	5	3	9
(e)	$\{0,1,3,2\}$	0	1	5	3	6
(f)	$\{0,1,3,2,4\}$	0	1	5	3	6

Bibliografia

BOAVENTURA NETTO, Paulo Oswaldo; JURKIEWICZ, Samuel.
Grafos: introdução e prática. Editora Blucher. 2009. ISBN: 9788521204732.

[Cormen et al., 2009] Introduction to Algorithms, 3rd edition.
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein.
2009, 1312 pp. MIT Press, ISBN 9780262033848.

[Ziviani, 2006] Projeto de Algoritmos com Implementações em Java e C++.
Nivio Ziviani. 2006, 642 pp. Editora Thomson, ISBN 8522105251.